GUIDE 37 · OPERATE

Diagnose and tune the connection pool

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=37-tune-the-connection-pool. This copy was made on 4 November 2026.

Time: 15 minutes. **You'll end up with:** an understanding of how many connections QueryAPIGate opens to your database, numbers to check that against, and the two settings that change it.

How it works

For PostgreSQL, MySQL, ClickHouse, DuckDB, H2 and JDBC connections, QueryAPIGate keeps database connections open after a request and reuses them for the next one, instead of connecting every time. (SQLite files are opened per request - there's nothing to pool.)

SETTING	DEFAULT	MEANING
`QUERYAPIGATE_POOL_SIZE`	`8`	idle connections kept **per connection** (per distinct set of connection details); `0` turns pooling off
`QUERYAPIGATE_POOL_IDLE_TIMEOUT`	`300`	seconds an idle connection may wait for reuse before it's closed

The important detail: **the pool limits idle connections, not concurrent ones.** Each request in progress uses its own connection; how many run at once is limited by the server's threads - 8 in the Docker image's gunicorn.

What it looks like - measured

QueryAPIGate under gunicorn as the Docker image runs it (1 worker, 8 threads), against a PostgreSQL 16 container. 20 queries of `SELECT pg_sleep(0.5)` sent at once; connections counted in PostgreSQL's `pg_stat_activity`:

	DURING THE BURST	AFTERWARDS (IDLE)	`SELECT 1`, MEDIAN
`QUERYAPIGATE_POOL_SIZE=5`	8	5	1.8 ms
`QUERYAPIGATE_POOL_SIZE=0`	8	0	6.6 ms

- During the burst, **8** connections - one per thread; the other 12 requests waited for a thread, not a connection.
- Afterwards, the pool kept **5** and closed the other 3. With the default of 8 - one per thread - it keeps all 8, so a busy server never closes a connection only to open another for the next request.
- Without the pool, every request paid for a new connection: about 5 ms more here, on the same machine. Over a network, with TLS and authentication, it's typically tens of milliseconds.

`/metrics` shows the idle count - `queryapigate_pool_idle_connections 5` after the burst - and the Grafana dashboard has a panel for it (guide 35).

Idle timeout: lazy, not swept

With `QUERYAPIGATE_POOL_IDLE_TIMEOUT=4`, five idle connections were **still open 6 seconds later**. The next request closed the expired ones and reused one: `pg_stat_activity` then showed 1. Expired connections are closed when the pool is next used, not by a background timer - so a server that goes completely quiet keeps its idle connections until traffic returns. If your database or a firewall drops idle connections sooner than that, it's handled: a connection idle for more than a few seconds is checked before reuse, and a dead one is replaced.

Sizing it

The most connections QueryAPIGate holds to one database is roughly:

- **busy:** threads × processes (8 × 1 in the Docker image), plus one per streamed export in progress;
- **idle:** `QUERYAPIGATE_POOL_SIZE` × processes, per connection that points at that database.

Add `queryapigate mcp` and `queryapigate events` processes if they query the same database, and keep the total below the database's `max_connections` - leaving room for everything else that uses it.

Symptoms and what to change:

YOU SEE	LIKELY CAUSE	CHANGE
Latency rises with traffic, CPU is low, `queryapigate_active_queries` sits at the thread count	requests are queuing for threads	more threads, `QUERYAPIGATE_THREADS` (see Tune a deployment for throughput)
"too many connections" errors from the database	threads (or several QueryAPIGate processes) exceed its limit	fewer threads, a higher `max_connections`, or a pooler such as PgBouncer in front of the database
Every request pays connection setup; `queryapigate_pool_idle_connections` stays at 0	pooling is off, or traffic is so sparse that connections expire	default `QUERYAPIGATE_POOL_SIZE`, longer `QUERYAPIGATE_POOL_IDLE_TIMEOUT`
The DBA asks why connections sit idle all night	the lazy timeout above	`QUERYAPIGATE_POOL_SIZE=1` or `2` on that server, or accept it - idle connections cost little
Other queries slow down while large exports run	each `?stream=true` export holds a connection for the whole download	fewer concurrent exports, or more threads

Good to know

- **Changes to a connection drop its pool.** Saving, editing or deleting a connection closes its idle pooled connections at once, so new details (a password, a host) take effect on the next request.
- **A failed or timed-out query's connection is closed**, not handed to the next request.
- **Clean hand-over:** each connection's transaction ends before reuse, so a request never sees a stale snapshot or another request's settings.
- **Per process:** each gunicorn worker and each QueryAPIGate process has its own pool.

Next steps

- Wire up Prometheus and Grafana.
- Export a large result without running out of memory – streaming and connections.
- Connection pooling reference.